

Python for Web Development

Lesson 3: Working with Jinja2 Templates

Working with Jinja2 Templates - Study Notes

Key Concepts

1. **Control Structures** – Using `{% for %}` loops and `{% if %}` conditionals to render repeated or conditional HTML.
2. **Filters** – Applying built in (e.g., `| safe`, `| lower`, `| default`) and custom filters to transform data inside templates.
3. **Template Inheritance** – Leveraging `{% extends %}` and `{% block %}` to create reusable base layouts.
4. **Context Passing** – Supplying Python objects (dicts, lists, objects) to the template via `render()` and accessing their attributes/methods.
5. **Escaping & Security** – Understanding auto escaping, the `| safe` filter, and how to avoid XSS vulnerabilities.

Summary

Jinja2 is a powerful, Python based templating engine that lets you generate dynamic HTML by combining static markup with data supplied from your application. The core of Jinja2's dynamism lies in its control structures: `{% for %}` loops iterate over sequences, while `{% if %}` statements conditionally include or exclude blocks of markup. Filters act as inline functions that transform values—`{{ name|title }}` capitalizes a string, `{{ price|floatformat:2 }}` formats a number, and custom filters let you inject any Python logic you need.

Templates are typically organized using inheritance. A base template defines the overall page skeleton (doctype, header, footer, etc.) and declares named `{% block %}` sections. Child templates `{% extends "base.html" %}` fill those blocks, keeping your HTML DRY and maintainable. When rendering, you pass a *context* (a dictionary or object) to `template.render()`. Jinja2 automatically escapes variables to protect against XSS, but you can deliberately mark safe HTML with the `| safe` filter when you're certain the content is trusted.

Important Points

- **Syntax**
 - Statements: `{% ... %}` (loops, conditionals, blocks, imports)
 - Expressions: `{{ ... }}` (output variables, apply filters)
 - Comments: `{# ... #}`
- **Loop Controls**
 - `loop.index`, `loop.index0`, `loop.revindex`, `loop.first`, `loop.last` give you positional info.
 - Use `{% else %}` after a `{% for %}` to handle empty sequences.

- **Conditionals**
 - Support ``and`, `or`, `not`, `in`, `==`, `!=`, `<`, `>`, `<=`, `>=`.`
 - ``{% elif %}`` works like Python's ``elif``.
- **Filters**
 - Chain multiple filters: ``{{ value|trim|lower }}``.
 - Provide arguments: ``{{ price|floatformat:2 }}``.
 - Register custom filters with ``app.jinja_env.filters['myfilter'] = myfilter_func``.
- **Escaping**
 - Auto escaping is on for files with extensions like ``.html`, `.htm`, `.xml``.
 - Use ``| safe`` **only** on content you control or have sanitized.
- **Template Inheritance**
 - Base template defines ``{% block content %}{% endblock %}`` placeholders.
 - Child overrides with ``{% block content %}...{% endblock %}``.
 - ``{{ super() }}`` inserts the parent block's content.
- **Macros**
 - Reusable snippets: ``{% macro input(name, value="") %}<input name="{{ name }}" value="{{ value|e }}">{% endmacro %}``.
 - Call with ``{{ input('email') }}``.
- **Including Templates**
 - ``{% include "navbar.html" %}`` inserts another template at render time.
 - ``{% import "forms.html" as forms %}`` lets you call macros via ``{{ forms.input(...) }}``.

Examples

1b ã Loop + Conditional + Filter

Python (Flask view)

```
python
@app.route("/products")
def products():
    items = [
        {"name": "Laptop", "price": 999.99, "in_stock": True},
        {"name": "Mouse", "price": 25.5, "in_stock": False},
        {"name": "Keyboard", "price": 45, "in_stock": True},
    ]
    return render_template("products.html", items=items)
```

templates/products.html

```
jinja
{% extends "base.html" %}
{% block content %}
<h2>Product List</h2>
<ul>
```

```
{% for item in items %}
<li>
    {{ loop.index }}. {{ item.name|title }} – ${{ item.price|floatformat:2 }}
    {% if item.in_stock %}
        <span class="badge badge-success">In stock</span>
    {% else %}
        <span class="badge badge-danger">Out of stock</span>
    {% endif %}
</li>
{% else %}
    <li>No products available.</li>
{% endfor %}
</ul>
{% endblock %}
---
```

****Explanation****

- `{% for item in items %}` iterates over the list.
- `loop.index` gives a 1 based counter.
- `|title` capitalizes each word of the product name.
- `|floatformat:2` forces two decimal places for the price.
- The `{% if %}` block shows a different badge based on `in\_stock`.
- The `{% else %}` after the loop handles the empty list case.

2b ã Custom Filter & Macro

****Python – register filter****

```
```python
def currency(value, symbol="$"):
 """Format a number as currency."""
 return f'{symbol}{value:,.2f}'
app.jinja_env.filters['currency'] = currency

```

#### **\*\*`templates/base.html` (partial)\*\***

```
```jinja
{% macro alert(message, type="info") %}
<div class="alert alert-{{ type }}">
    {{ message|e }}
</div>
{% endmacro %}
---
```

****`templates/dashboard.html`****

```
```jinja
```

```
{% extends "base.html" %}
{% from "base.html" import alert %}
{% block content %}
<h1>Dashboard</h1>
{{ alert("Welcome back!", "success") }}
<p>Your balance: {{ balance|currency:"€" }}</p>
{% endblock %}

```

#### **\*\*Explanation\*\***

- The `currency` filter formats numbers with thousands separators and two decimals, optionally using a different symbol.
- The `alert` macro creates a reusable Bootstrap alert component; it's imported with `{% from ... import alert %}`.
- In the child template, `{% alert(...) %}` renders the macro, and `{% balance|currency:"€" %}` applies the custom filter.

---

## **Quick Review**

1. **\*\*What Jinja2 variable syntax automatically escapes HTML, and how do you deliberately bypass it?\*\***
2. **\*\*Name two loop attributes you can use inside a `{% for %}` block and describe their values.\*\***
3. **\*\*How would you pass a custom Python function to a template as a filter? Provide a one line code example.\*\***
4. **\*\*Explain the purpose of `{% block %}` and `{% super() %}` in template inheritance.\*\***
5. **\*\*What is the difference between `{% include %}` and `{% import %}`? When would you use each?\*\***

---

## **Further Reading**

- **\*\*Advanced Filters\*\*** – Writing complex filters, handling arguments, and registering them globally.
- **\*\*Template Sandbox\*\*** – Using Jinja2's sandboxed environment for untrusted templates.
- **\*\*Internationalization (i18n)\*\*** – Integrating `gettext` with Jinja2 for multilingual sites.
- **\*\*Performance Tips\*\*** – Template caching, bytecode compilation, and minimizing context size.
- **\*\*Testing Templates\*\*** – Unit testing Jinja2 output with tools like `pytest` and `jinja2 test`.

